CSA3714-SOFTWARE TESTING

SELENIUM LAB PROGRAMS

36. Write a Selenium automation script to open the Chrome browser and verify the title of ARMS portal.

AIM

To open the ARMS portal in the Chrome browser using Selenium WebDriver and verify the page title.

PROCEDURE

1. Import Selenium WebDriver and the time module.
2. Create a Chrome WebDriver instance and maximize the browser window.
3. Open the ARMS portal URL.
4. Retrieve and display the page title.
5. Keep the browser open for five minutes and close it.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path

# Create HTML webpage
html = """
<html>
<head>
    <title>Example Domain</title>
</head>
<body>
    <h2>Website Title Verification</h2>

    <label>Expected Title:</label>
    <input type="text" id="title" placeholder="Enter website title">

    <button onclick="
        let value = document.getElementById('title').value;

        if(value == document.title) {
            document.getElementById('result').innerHTML =
            'Test Passed! Website title is correct.';
        } else {
            document.getElementById('result').innerHTML =
            'Test Failed! Website title is incorrect.';
        }
    ">Verify Title</button>

    <p id="result"></p>
</body>
</html>
"""

# Save HTML file
```

1931363

```
file = Path("website_title.html")
file.write_text(html, encoding="utf-8")

# Open Chrome
driver = webdriver.Chrome()

try:
    driver.get(file.resolve().as_uri())

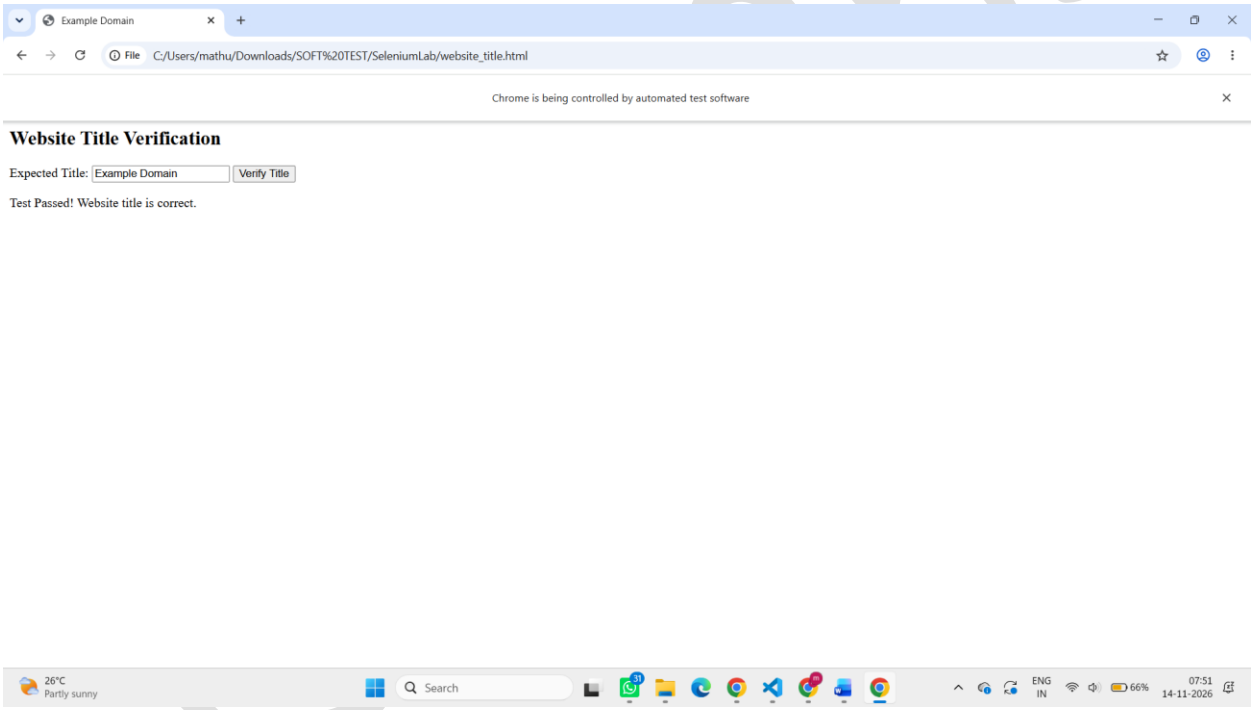
    # Enter input
    title = driver.find_element(By.ID, "title")
    title.send_keys("Example Domain")

    # Click button
    driver.find_element(By.TAG_NAME, "button").click()

    input("Press Enter to close browser...")

finally:
    driver.quit()
```

OUTPUT



RESULT

Thus, the Chrome browser was opened using Selenium and the title of the ARMS portal was retrieved and displayed.

37. Write a Selenium automation script to automate search functionality in Google using the Chrome browser.

AIM

To automate Google search using Selenium WebDriver in the Chrome browser.

PROCEDURE

1. Import the required Selenium modules and time module.
2. Launch Chrome and open the Google website.
3. Locate the search box using its name attribute.
4. Enter the search keyword and press the Enter key.
5. Display the resulting page title, wait for five minutes, and close the browser.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# Open Chrome browser
driver = webdriver.Chrome()

try:
    # Open Google
    driver.get("https://www.google.com")

    # Enter search input
    search_box = WebDriverWait(driver, 15).until(
        EC.presence_of_element_located((By.NAME, "q"))
    )
```

1923136

```
    search_box.send_keys("Saveetha School of Engineering")

    # Perform search
    search_box.send_keys(Keys.RETURN)

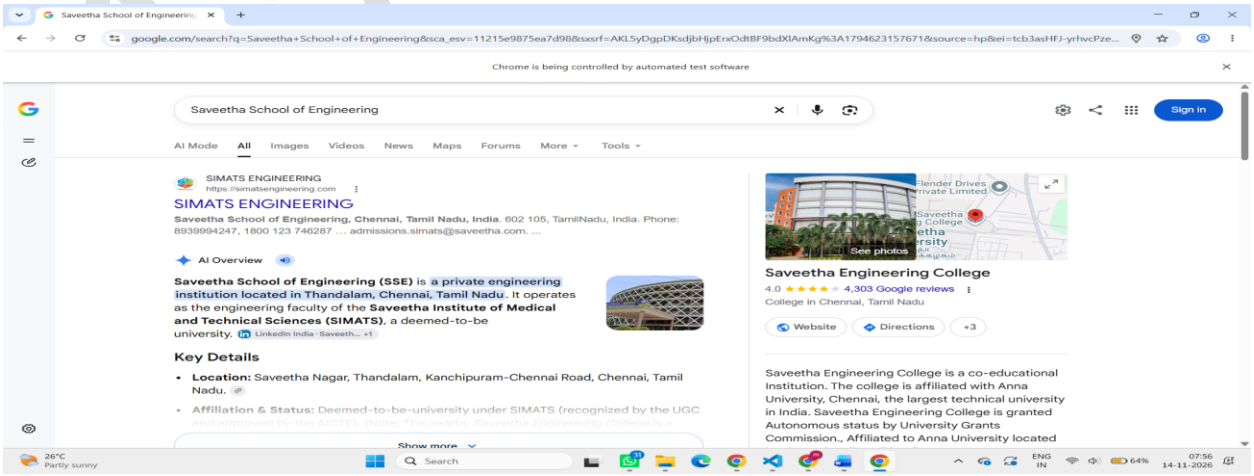
    # Wait for search results
    WebDriverWait(driver, 15).until(
        EC.presence_of_element_located((By.ID, "search"))
    )

    print("Google Search Completed Successfully")
    print("Search Input: Saveetha School of Engineering")
    print("Page Title:", driver.title)

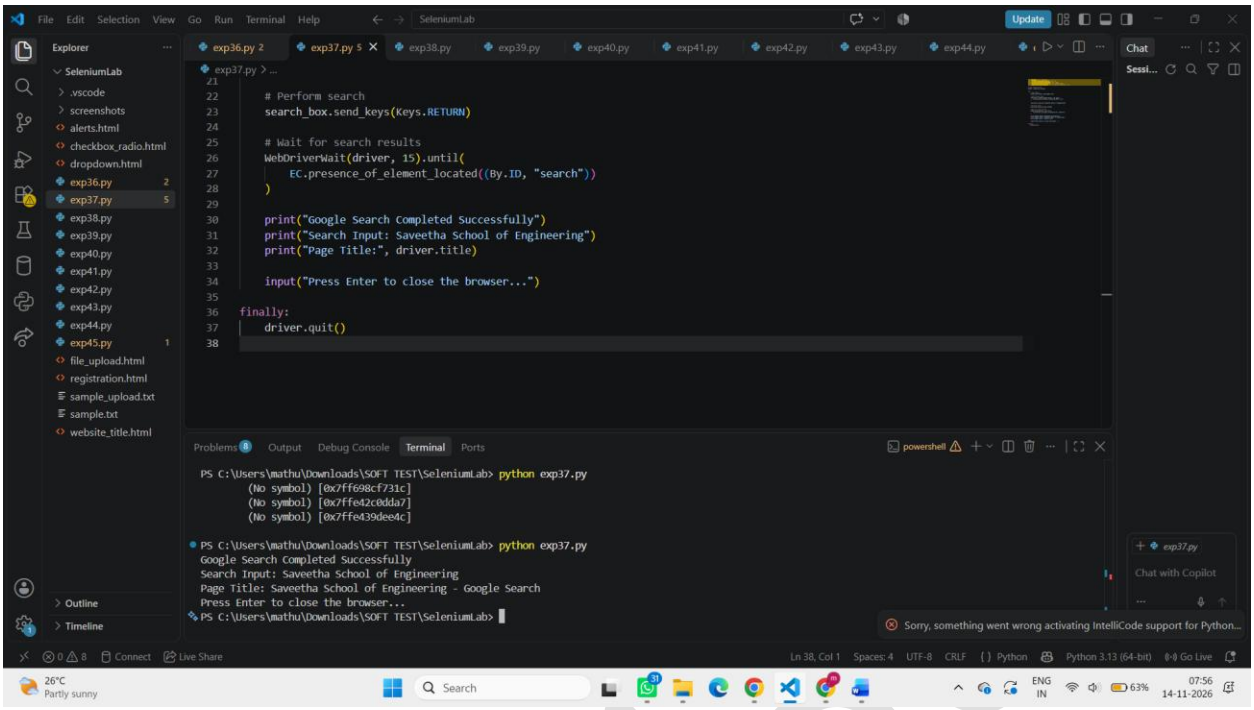
    input("Press Enter to close the browser...")

finally:
    driver.quit()
```

OUTPUT



92231363



RESULT

Thus, Google search functionality was automated successfully using Selenium WebDriver.

192115

38. Write a Selenium program to automate user registration form filling (enter name, email, password, and submit) using the Chrome browser.

AIM

To automate filling and submission of a user registration form using Selenium WebDriver.

PROCEDURE

1. Import Selenium WebDriver, By, and time modules.
2. Launch Chrome and open the registration practice page.
3. Locate the name, email, password, and confirm-password fields.
4. Enter the registration details in the respective fields.
5. Click the Submit button and display the resulting page heading.
6. Keep Chrome open for five minutes and close the browser.

PROGRAM

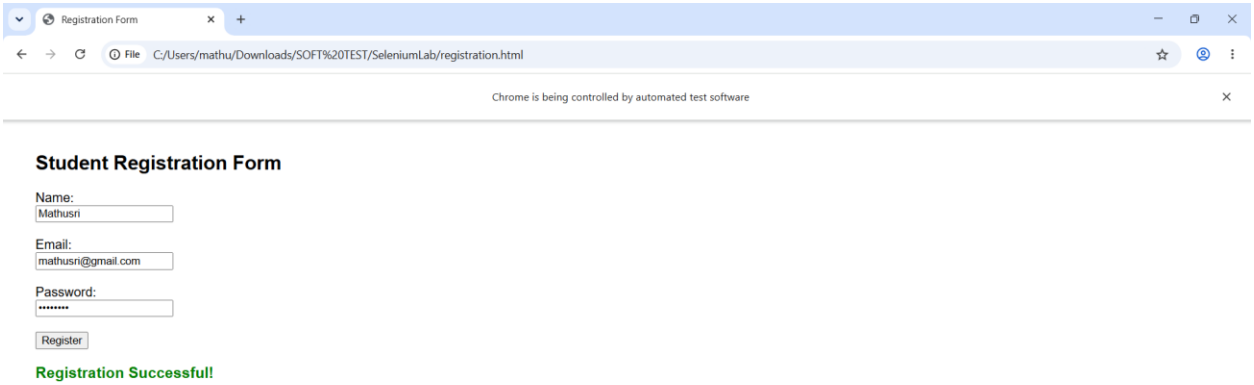
```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path
# Create registration webpage
html = """
<html>
<head>
  <title>Registration Form</title>
</head>
<body style="font-family:Arial; margin:40px;">
  <h2>Student Registration Form</h2>
  <label>Name:</label><br>
  <input type="text" id="name" placeholder="Enter your name"><br><br>
  <label>Email:</label><br>
  <input type="email" id="email" placeholder="Enter your email"><br><br>
  <label>Password:</label><br>
  <input type="password" id="password" placeholder="Enter password"><br><br>
```

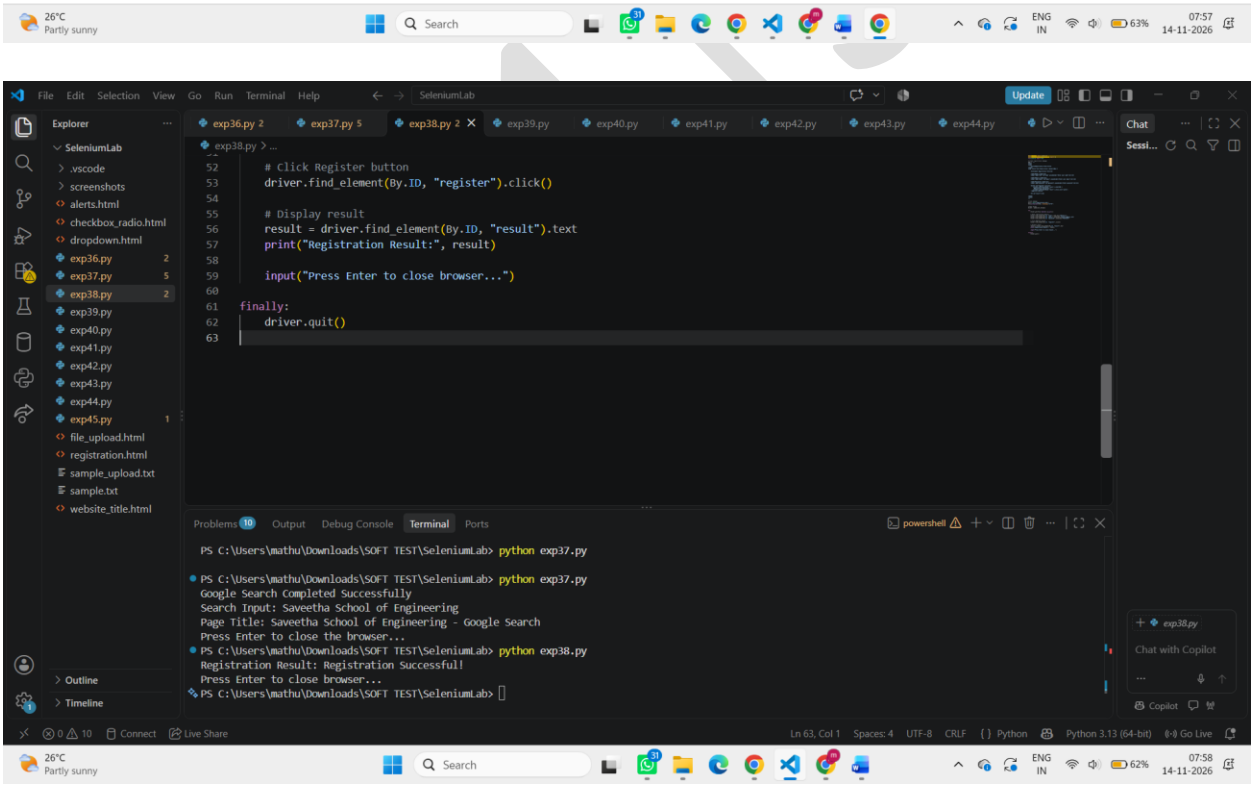
1923136

```
    <button id="register" onclick="
        document.getElementById('result').innerHTML =
        'Registration Successful!';
        document.getElementById('result').style.color='green';
    ">Register</button>
    <h3 id="result"></h3>
</body>
</html>
"""
# Save webpage
file = Path("registration.html")
file.write_text(html, encoding="utf-8")
# Open Chrome
driver = webdriver.Chrome()
try:
    driver.get(file.resolve().as_uri())
    # Enter registration details
    driver.find_element(By.ID, "name").send_keys("Mathusri")
    driver.find_element(By.ID, "email").send_keys("mathusri@gmail.com")
    driver.find_element(By.ID, "password").send_keys("Mathu123")
    # Click Register button
    driver.find_element(By.ID, "register").click()
    # Display result
    result = driver.find_element(By.ID, "result").text
    print("Registration Result:", result)
    input("Press Enter to close browser...")
finally:
    driver.quit()
```

1923136363

OUTPUT





RESULT

Thus, the user registration form was filled and submitted using Selenium WebDriver.

39. Write a Selenium automation script to automate login into ARMS portal and validate error message for invalid credentials.

AIM

To automate login to the ARMS portal with invalid credentials and validate the displayed error message.

PROCEDURE

1. Import Selenium WebDriver, locator, wait, and time modules.
2. Launch Chrome and open the ARMS portal login page.
3. Enter an invalid username and password.
4. Click the Login button.
5. Wait for the error message and display it.
6. Keep Chrome open for five minutes and close the browser.

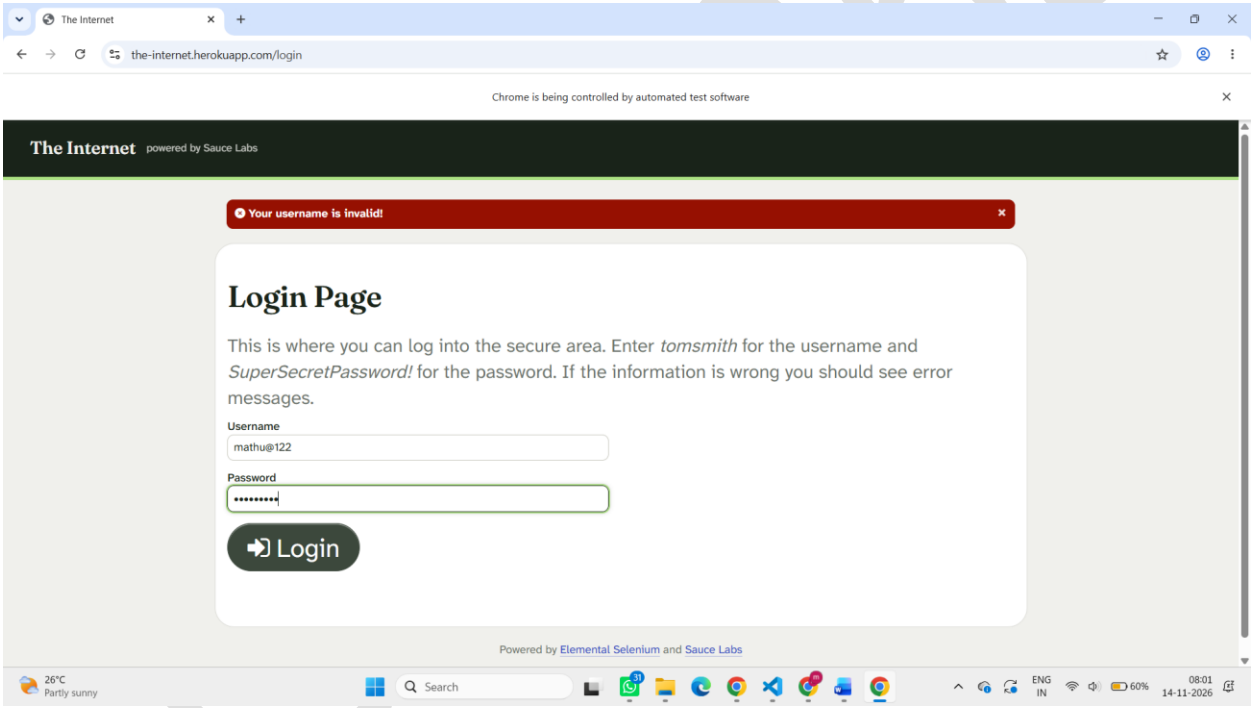
PROGRAM

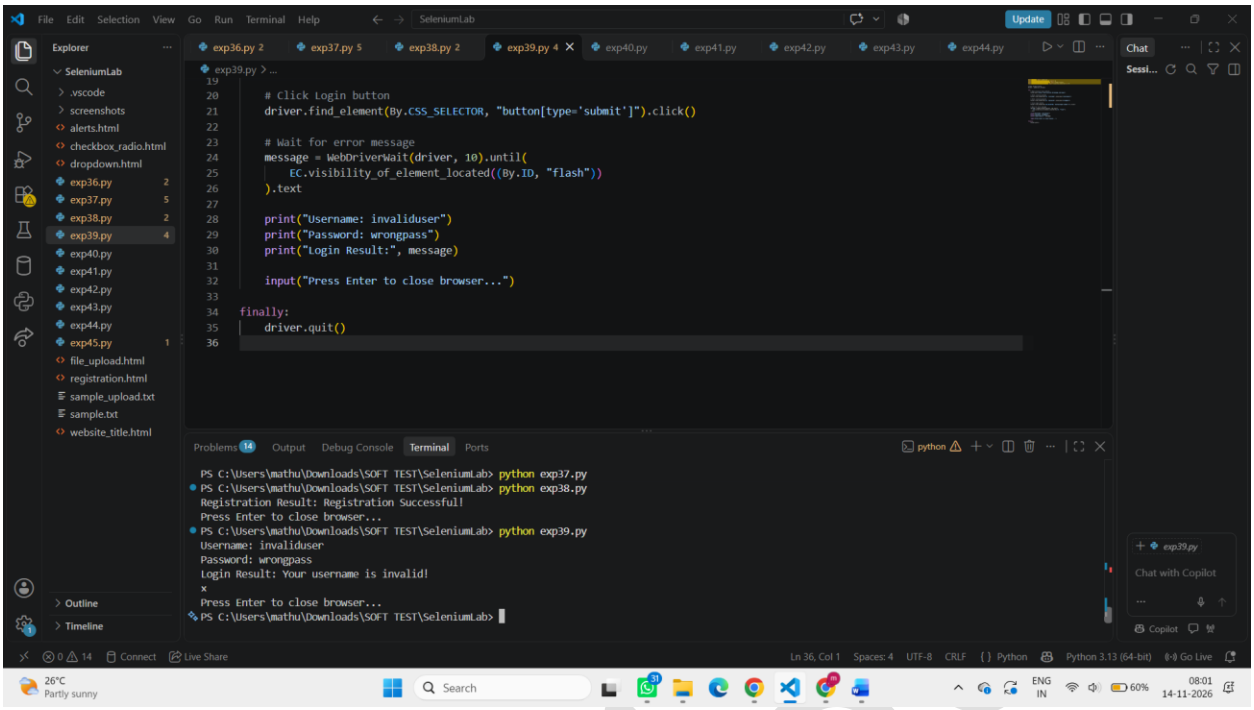
```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
# Open Chrome browser
driver = webdriver.Chrome()
try:
    # Open practice login website
    driver.get("https://the-internet.herokuapp.com/login")
    # Enter invalid username
    driver.find_element(By.ID, "username").send_keys("invaliduser")
    # Enter invalid password
    driver.find_element(By.ID, "password").send_keys("wrongpass")
    # Click Login button
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    # Wait for error message
    message = WebDriverWait(driver, 10).until(
```

1931363

```
        EC.visibility_of_element_located((By.ID, "flash"))
    ).text
    print("Username: invaliduser")
    print("Password: wrongpass")
    print("Login Result:", message)
    input("Press Enter to close browser...")
finally:
    driver.quit()
```

OUTPUT





RESULT

Thus, the login process with invalid credentials was automated and the error message was checked. The ARMS page must be accessible, and its field locators may need to be adjusted to match the live portal.

192115

40. Write a Selenium automation script to automate logout functionality from a web application using the Chrome browser.

AIM

To automate the logout functionality of a web application using Selenium WebDriver.

PROCEDURE

1. Import the required Selenium modules and time module.
2. Launch Chrome and open the login page of the practice web application.
3. Enter valid test credentials and click Login.
4. Wait until the secure page is displayed.
5. Locate and click the Logout link.
6. Verify that the browser returns to the login page.

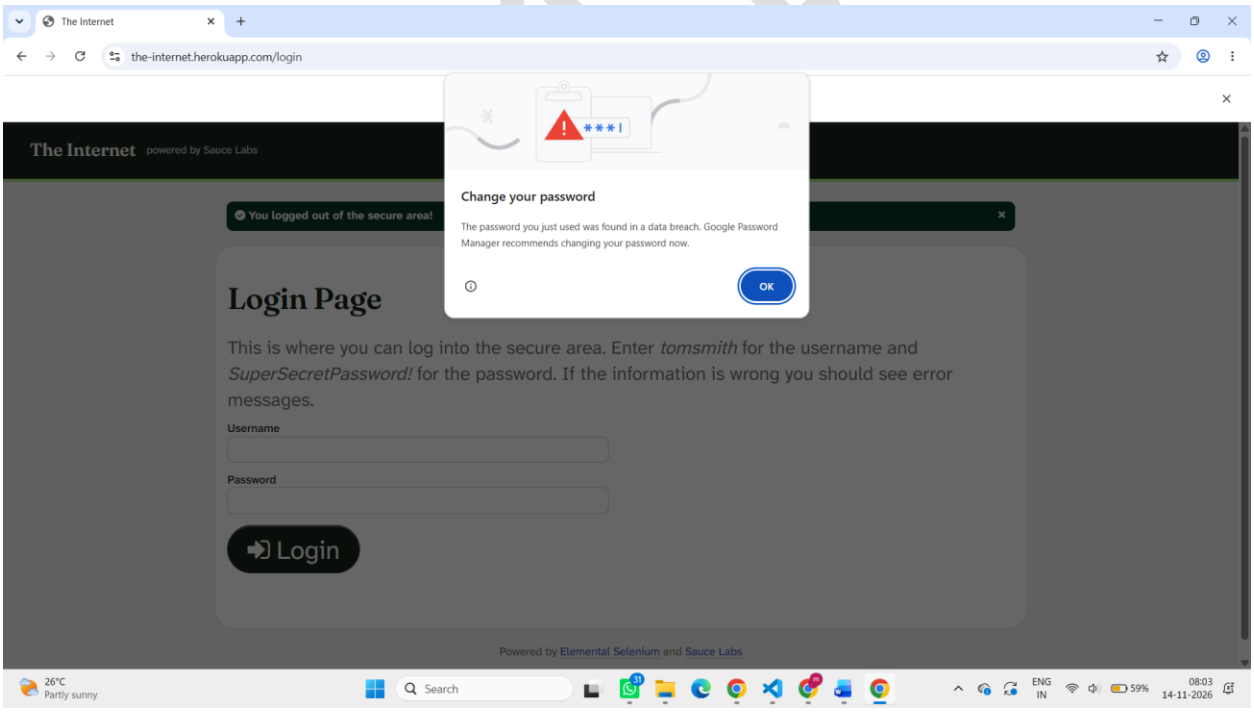
PROGRAM

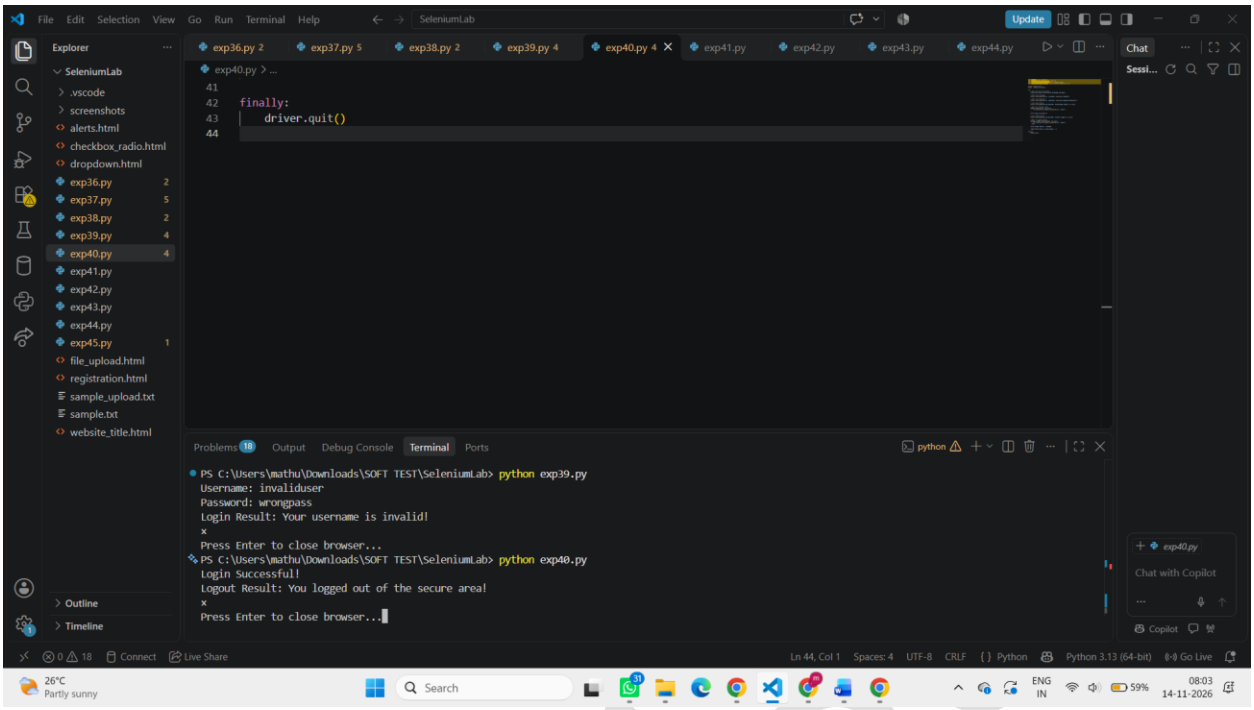
```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
# Open Chrome browser
driver = webdriver.Chrome()
try:
    # Open practice login website
    driver.get("https://the-internet.herokuapp.com/login")
    # Enter valid username
    driver.find_element(By.ID, "username").send_keys("tomsmith")
    # Enter valid password
    driver.find_element(By.ID, "password").send_keys("SuperSecretPassword!")
    # Click Login button
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    # Wait for successful login
    WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located((By.ID, "flash"))
```

1923163

```
    )
    print("Login Successful!")
    # Click Logout button
    driver.find_element(By.CSS_SELECTOR, "a[href='/logout']").click()
    # Wait for logout message
    message = WebDriverWait(driver, 10).until(
        EC.visibility_of_element_located((By.ID, "flash"))
    ).text
    print("Logout Result:", message)
    input("Press Enter to close browser...")
finally:
    driver.quit()
```

OUTPUT





RESULT

Thus, the logout functionality was automated and successful redirection to the login page was verified.

1923153

41. Write a Selenium automation script to automate dropdown selection (using selectByVisibleText) on a web application using the Chrome browser.

AIM

To select an option from a dropdown list using Selenium WebDriver and the Select class.

PROCEDURE

1. Import WebDriver, By, Select, and time modules.
2. Launch Chrome and open the dropdown practice page.
3. Locate the dropdown element.
4. Create a Select object for the dropdown.
5. Select the required option using visible text and display the selected option.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path
# Create HTML webpage
html = """
<html>
<head>
   <title>Dropdown Selection</title>
</head>
<body style="font-family:Arial; margin:40px;">
   <h2>Car Selection Form</h2>
   <label>Select Your Car:</label><br><br>
   <select id="cars" onchange="showResult()">
      <option value="">Choose a car</option>
      <option value="BMW">BMW</option>
      <option value="Audi">Audi</option>
      <option value="Benz">Benz</option>
      <option value="Tesla">Tesla</option>
```

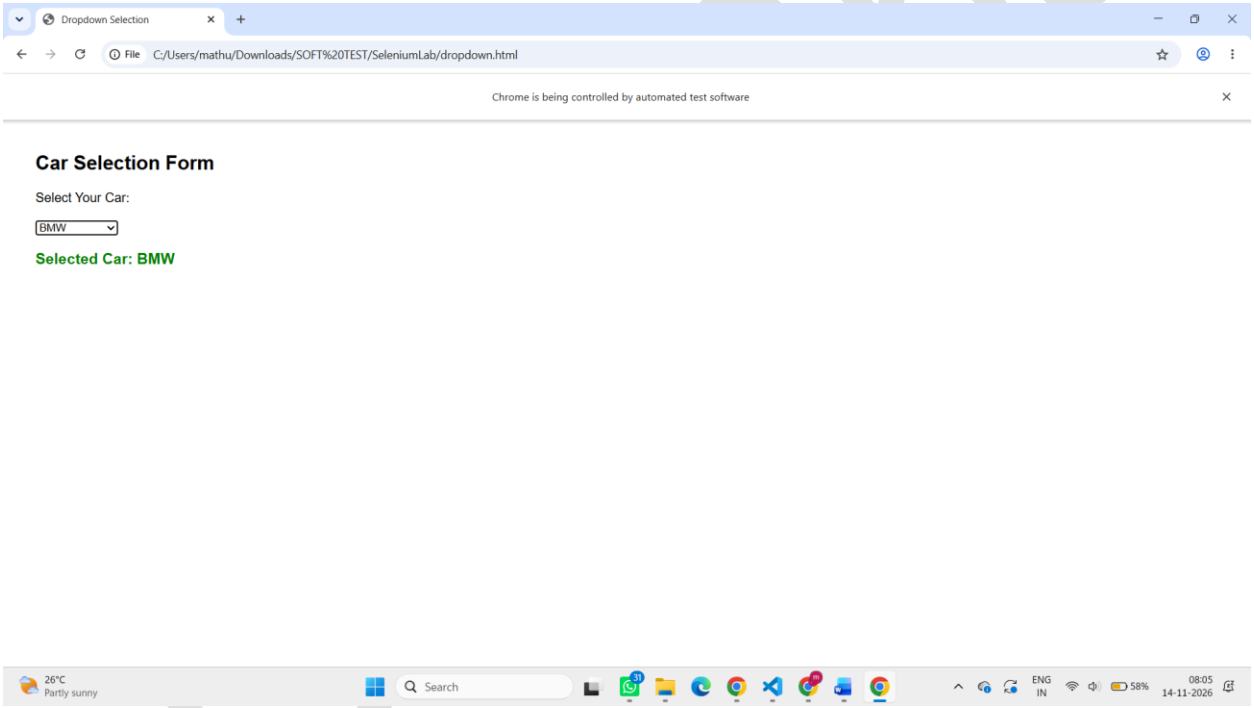
1923136

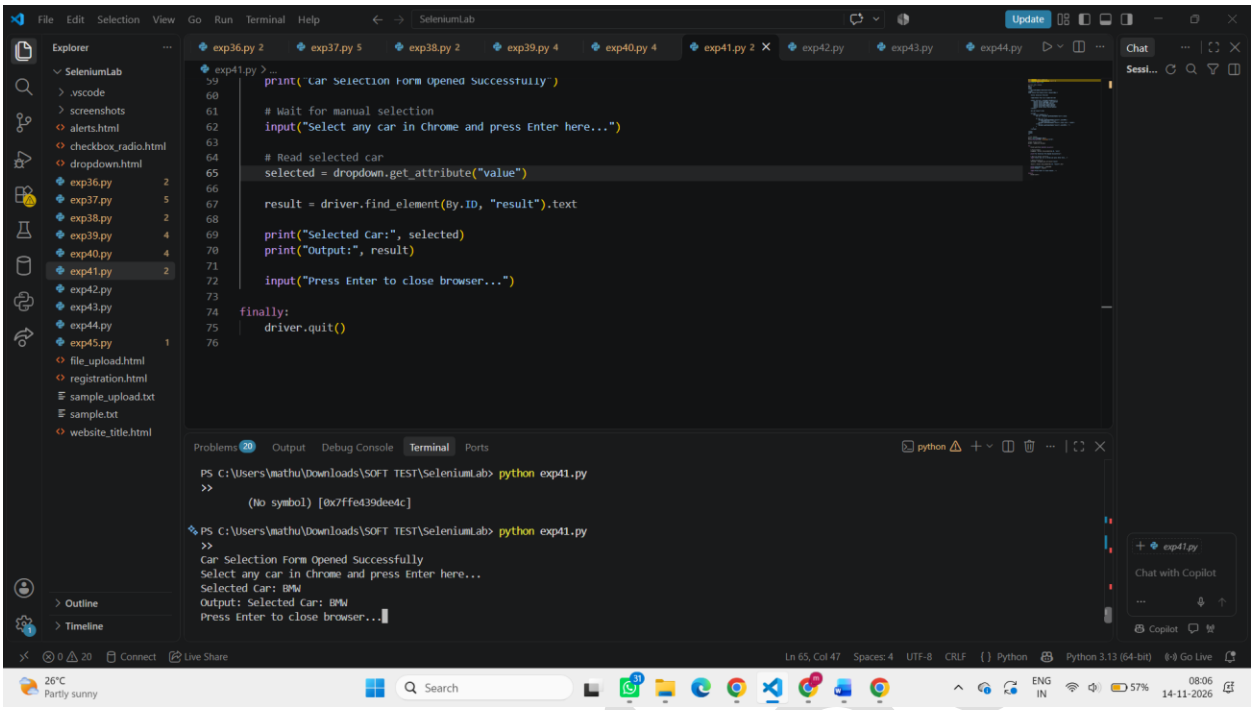
```
    </select>
    <h3 id="result"></h3>
    <script>
        function showResult() {
            let car = document.getElementById("cars").value;
            if (car !== "") {
                document.getElementById("result").innerHTML =
                    "Selected Car: " + car;
                document.getElementById("result").style.color = "green";
            } else {
                document.getElementById("result").innerHTML = "";
            }
        }
    </script>
</body>
</html>
"""
# Save webpage
file = Path("dropdown.html")
file.write_text(html, encoding="utf-8")
# Open Chrome browser
driver = webdriver.Chrome()
try:
    driver.get(file.resolve().as_uri())
    # Find dropdown
    dropdown = driver.find_element(By.ID, "cars")
    print("Car Selection Form Opened Successfully")
    # Wait for manual selection
    input("Select any car in Chrome and press Enter here...")
```

1923163

```
    # Read selected car
    selected = dropdown.get_attribute("value")
    result = driver.find_element(By.ID, "result").text
    print("Selected Car:", selected)
    print("Output:", result)
    input("Press Enter to close browser...")
finally:
    driver.quit()
```

OUTPUT





RESULT

Thus, the required dropdown option was selected successfully using the Select class and visible text.

1923153

42. Write a Selenium program to automate checkbox and radio button selection on a form using the Chrome browser.

AIM

To select a checkbox and a radio button on a web form using Selenium WebDriver.

PROCEDURE

1. Import Selenium WebDriver, By, and time modules.
2. Launch Chrome and open the Selenium web form page.
3. Locate the checkbox and check whether it is already selected.
4. Click the checkbox if it is not selected.
5. Locate and click the radio button.
6. Display the selection status of both elements.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path
# Create HTML webpage
html = """
<html>
<head>
   <title>Checkbox and Radio Button</title>
</head>
<body style="font-family:Arial; margin:40px;">
   <h2>Student Preference Form</h2>
   <h3>Select Your Hobbies:</h3>
   <input type="checkbox" id="reading" value="Reading"
   onchange="showResult()"> Reading<br>
   <input type="checkbox" id="music" value="Music"
   onchange="showResult()"> Music<br>
   <input type="checkbox" id="sports" value="Sports"
   onchange="showResult()"> Sports<br>
```

1923136

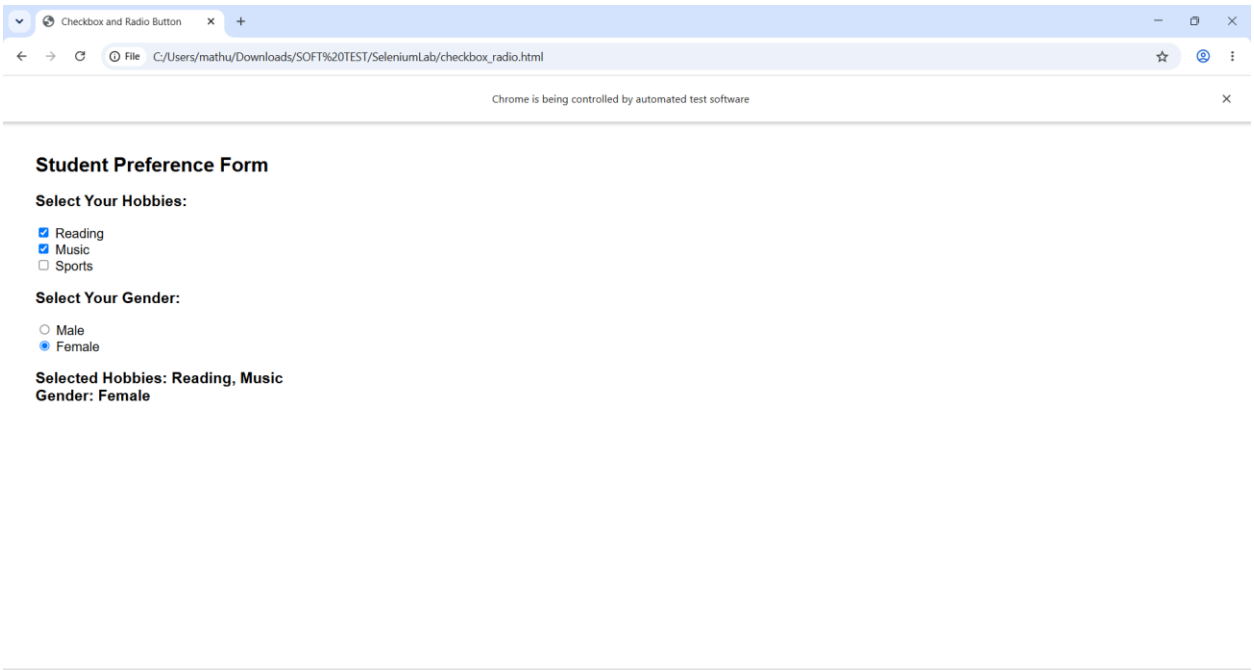
```
    <h3>Select Your Gender:</h3>
    <input type="radio" name="gender" value="Male"
    onchange="showResult()"> Male<br>
    <input type="radio" name="gender" value="Female"
    onchange="showResult()"> Female<br>
    <h3 id="result"></h3>
    <script>
        function showResult() {
            let hobbies = [];
            document.querySelectorAll(
                'input[type="checkbox"]:checked'
            ).forEach(item => hobbies.push(item.value));
            let gender = document.querySelector(
                'input[name="gender"]:checked'
            );
            let genderValue = gender ? gender.value : "Not selected";
            document.getElementById("result").innerHTML =
                "Selected Hobbies: " +
                (hobbies.length ? hobbies.join(", ") : "None") +
                "<br>Gender: " + genderValue;
        }
    </script>
</body>
</html>
"""
# Save webpage
file = Path("checkbox_radio.html")
file.write_text(html, encoding="utf-8")
# Open Chrome browser
```

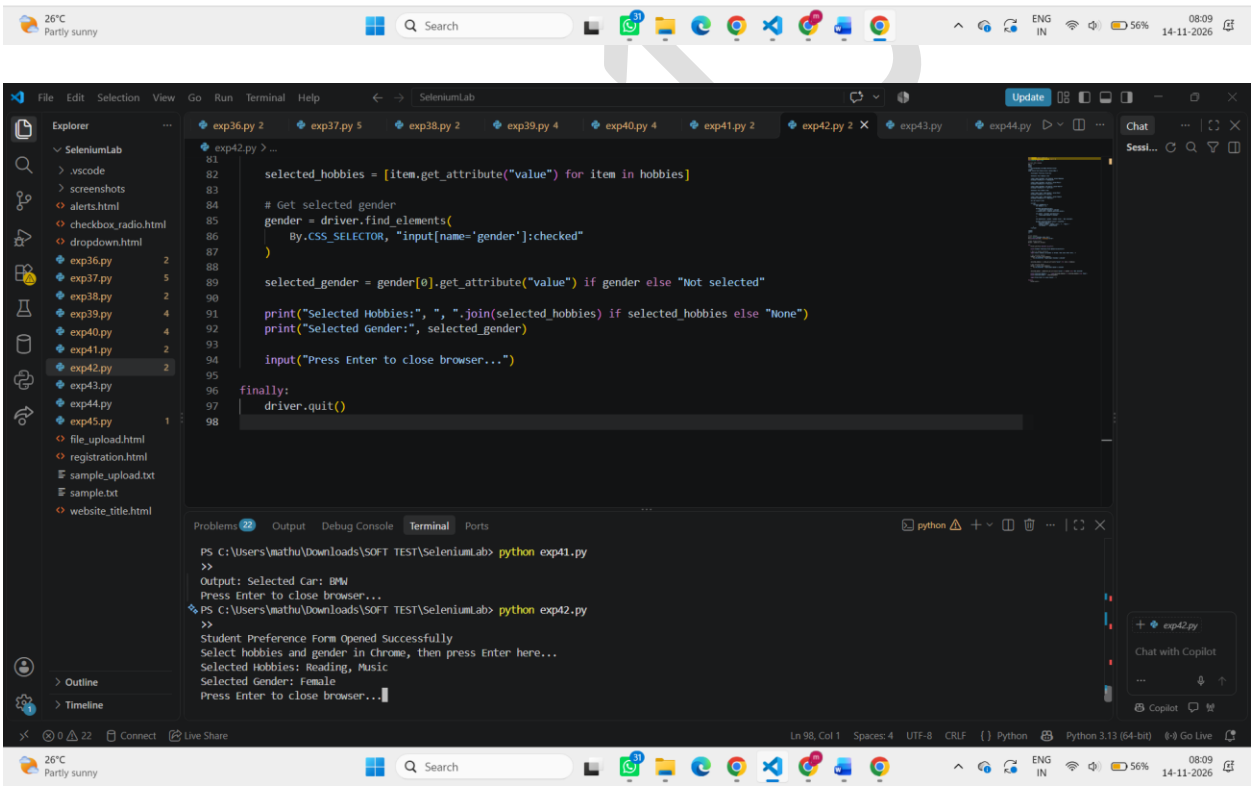
1921363

```
driver = webdriver.Chrome()
try:
    driver.get(file.resolve().as_uri())
    print("Student Preference Form Opened Successfully")
    # Wait for manual selection
    input("Select hobbies and gender in Chrome, then press Enter here...")
    # Get selected hobbies
    hobbies = driver.find_elements(
        By.CSS_SELECTOR, "input[type='checkbox']:checked"
    )
    selected_hobbies = [item.get_attribute("value") for item in hobbies]
    # Get selected gender
    gender = driver.find_elements(
        By.CSS_SELECTOR, "input[name='gender']:checked"
    )
    selected_gender = gender[0].get_attribute("value") if gender else "Not selected"
    print("Selected Hobbies:", ", ".join(selected_hobbies) if selected_hobbies else "None")
    print("Selected Gender:", selected_gender)
    input("Press Enter to close browser...")
finally:
    driver.quit()
```

OUTPUT

1921363





RESULT

Thus, the checkbox and radio button were selected and their selection status was verified using Selenium.

43. Write a Selenium automation script to handle JavaScript alert, confirm, and prompt pop-ups using the Chrome browser.

AIM

To handle JavaScript alert, confirmation, and prompt pop-up windows using Selenium WebDriver.

PROCEDURE

1. Import Selenium WebDriver, By, WebDriverWait, expected conditions, and time modules.
2. Launch Chrome and open the JavaScript alerts practice page.
3. Click the JavaScript Alert button, read the message, and accept the alert.
4. Click the JavaScript Confirm button, read the message, and accept the confirmation.
5. Click the JavaScript Prompt button, enter text, and accept the prompt.
6. Display the messages and keep Chrome open for five minutes.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path
# Create HTML webpage
html = """
<html>
<head>
   <title>JavaScript Alerts</title>
</head>
<body style="font-family:Arial; margin:40px;">
   <h2>JavaScript Alert Handling</h2>
   <button id="alert" onclick="alert('This is an Alert Box!')">
      Alert
   </button>
   <br><br>
   <button id="confirm"
   onclick="confirm('Do you want to continue?')">
```

1923136

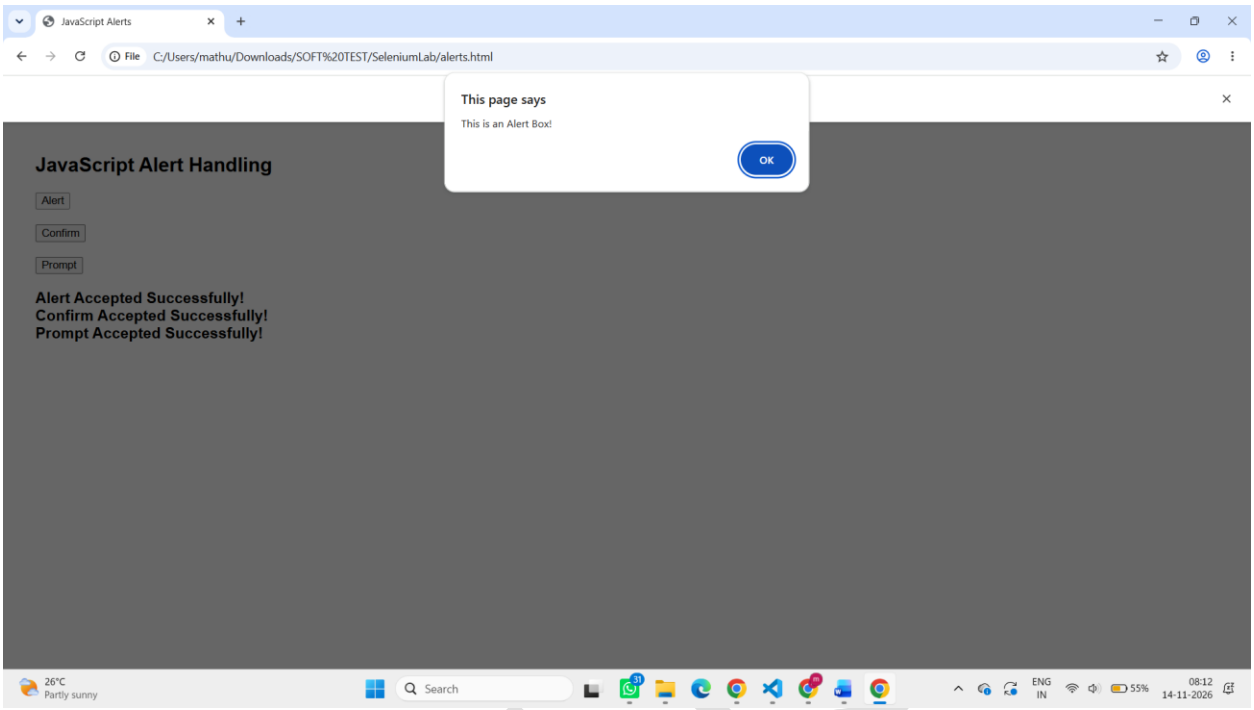
```
    Confirm
  </button>
  <br><br>
  <button id="prompt"
  onclick="prompt('Enter your name:')">
    Prompt
  </button>
  <h3 id="result">Results will appear here</h3>
</body>
</html>
"""
# Save webpage
file = Path("alerts.html")
file.write_text(html, encoding="utf-8")
# Open Chrome browser
driver = webdriver.Chrome()
try:
  driver.get(file.resolve().as_uri())
  # Handle Alert box
  driver.find_element(By.ID, "alert").click()
  alert = driver.switch_to.alert
  print("Alert Message:", alert.text)
  alert.accept()
  driver.execute_script("""
    document.getElementById('result').innerHTML =
    'Alert Accepted Successfully!';
  """)
  # Handle Confirm box
  driver.find_element(By.ID, "confirm").click()
```

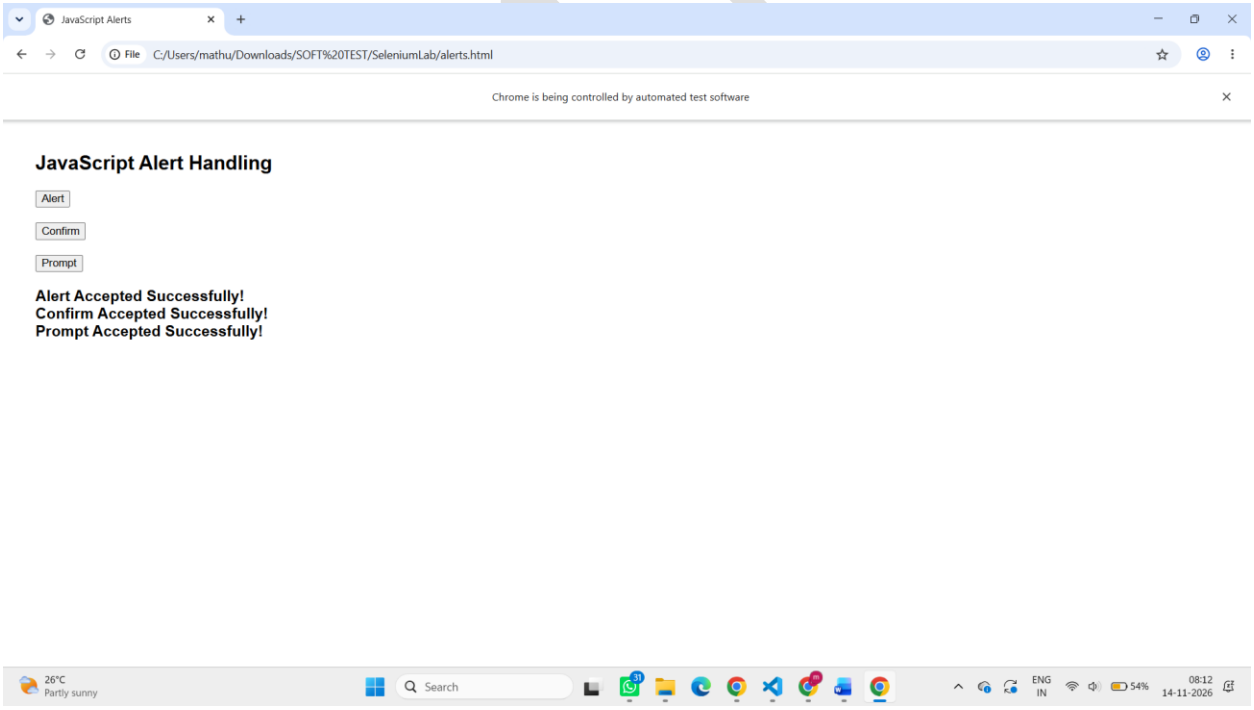
1923136363

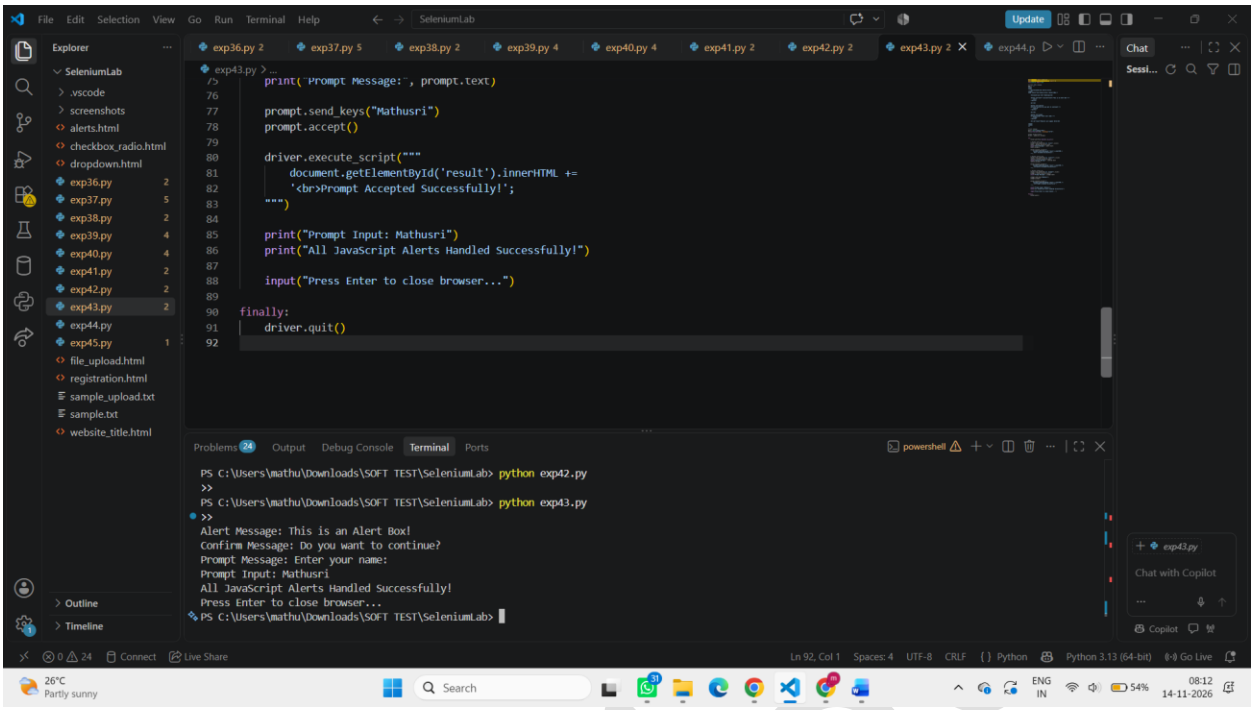
```
    confirm = driver.switch_to.alert
    print("Confirm Message:", confirm.text)
    confirm.accept()
    driver.execute_script("""
        document.getElementById('result').innerHTML +=
        '<br>Confirm Accepted Successfully!';
    """)
    # Handle Prompt box
    driver.find_element(By.ID, "prompt").click()
    prompt = driver.switch_to.alert
    print("Prompt Message:", prompt.text)
    prompt.send_keys("Mathusri")
    prompt.accept()
    driver.execute_script("""
        document.getElementById('result').innerHTML +=
        '<br>Prompt Accepted Successfully!';
    """)
    print("Prompt Input: Mathusri")
    print("All JavaScript Alerts Handled Successfully!")
    input("Press Enter to close browser...")
finally:
    driver.quit()
```

192231363

OUTPUT


JavaScript Alerts
File C:/Users/mathu/Downloads/SOFT%20TEST/SeleniumLab/alerts.html
This page says
This is an Alert Box!
OK
JavaScript Alert Handling
Alert
Confirm
Prompt
Alert Accepted Successfully!
Confirm Accepted Successfully!
Prompt Accepted Successfully!
26°C
Partly sunny
Search
ENG
IN
55%
08:12
14-11-2026


JavaScript Alerts
File C:/Users/mathu/Downloads/SOFT%20TEST/SeleniumLab/alerts.html
Chrome is being controlled by automated test software
JavaScript Alert Handling
Alert
Confirm
Prompt
Alert Accepted Successfully!
Confirm Accepted Successfully!
Prompt Accepted Successfully!
26°C
Partly sunny
Search
ENG
IN
54%
08:12
14-11-2026



RESULT

Thus, JavaScript alert, confirm, and prompt pop-ups were handled successfully using Selenium WebDriver.

1923153

44. Write a Selenium program to automate file upload functionality on a web application using the Chrome browser.

AIM

To upload a local file to a web application using Selenium WebDriver.

PROCEDURE

1. Import Selenium WebDriver, By, os, and time modules.
2. Create a sample text file in the same folder as the Python program.
3. Launch Chrome and open the file upload practice page.
4. Get the absolute path of the sample file.
5. Locate the file input element and pass the file path using send_keys().
6. Click the Upload button and verify the displayed result.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path

# Create sample file
sample_file = Path("sample_upload.txt").resolve()
sample_file.write_text("This is a sample file for Selenium upload.")

# Create HTML webpage
html = """
<html>
<head>
   <title>File Upload</title>
</head>
<body style="font-family:Arial; margin:40px;">

   <h2>File Upload Form</h2>
```

1923163

```
    <label>Select File:</label><br><br>

    <input type="file" id="file"><br><br>

    <button id="upload" onclick="
        let file = document.getElementById('file').files[0];

        if(file) {
            document.getElementById('result').innerHTML =
            'File Uploaded Successfully!<br>File Name: ' + file.name;

            document.getElementById('result').style.color = 'green';
        } else {
            document.getElementById('result').innerHTML =
            'Please select a file!';
            document.getElementById('result').style.color = 'red';
        }
    ">Upload</button>

    <h3 id="result"></h3>

</body>
</html>
"""

# Save webpage
webpage = Path("file_upload.html")
webpage.write_text(html, encoding="utf-8")
```

1923163

```
# Open Chrome browser
driver = webdriver.Chrome()

try:
    driver.get(webpage.resolve().as_uri())

    # Select file using Selenium
    driver.find_element(By.ID, "file").send_keys(str(sample_file))

    # Click Upload button
    driver.find_element(By.ID, "upload").click()

    # Get upload result
    result = driver.find_element(By.ID, "result").text

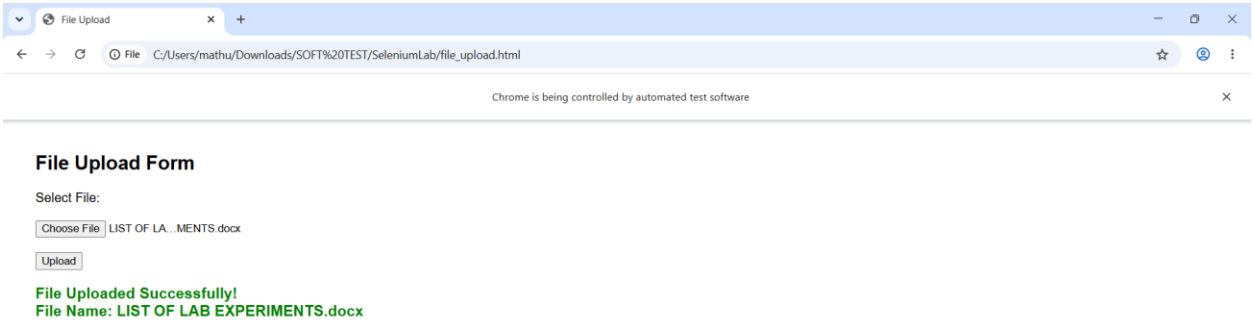
    print("Selected File:", sample_file.name)
    print("Upload Result:", result)

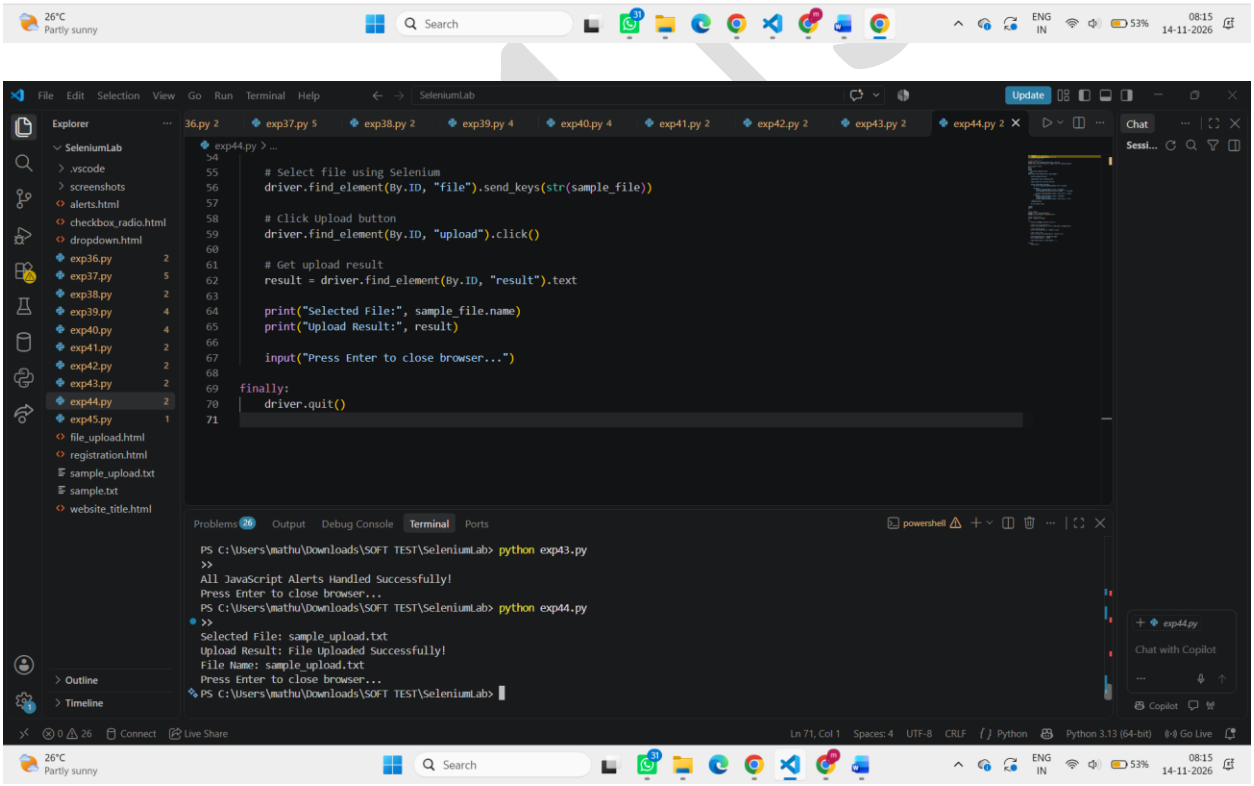
    input("Press Enter to close browser...")

finally:
    driver.quit()
```

192113 63

OUTPUT





RESULT

Thus, the selected local file was uploaded to the web application and the upload result was verified.

45. Write a Selenium automation script to capture a screenshot of a web page and store it in the local system using the Chrome browser.

AIM

To capture a screenshot of a web page and save it to the local system using Selenium WebDriver.

PROCEDURE

1. Import Selenium WebDriver, os, and time modules.
2. Launch Chrome and open the required web page.
3. Create the absolute path for the screenshot file.
4. Use save_screenshot() to capture and save the browser page.
5. Display the saved screenshot path.
6. Keep Chrome open for five minutes and close the browser.

PROGRAM

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from pathlib import Path

# Create sample file
sample_file = Path("sample_upload.txt").resolve()
sample_file.write_text("This is a sample file for Selenium upload.")

# Create HTML webpage
html = """
<html>
<head>
    <title>File Upload</title>
</head>
<body style="font-family:Arial; margin:40px;">

    <h2>File Upload Form</h2>
```

1923136

```
    <label>Select File:</label><br><br>

    <input type="file" id="file"><br><br>

    <button id="upload" onclick="
        let file = document.getElementById('file').files[0];

        if(file) {
            document.getElementById('result').innerHTML =
            'File Uploaded Successfully!<br>File Name: ' + file.name;

            document.getElementById('result').style.color = 'green';
        } else {
            document.getElementById('result').innerHTML =
            'Please select a file!';
            document.getElementById('result').style.color = 'red';
        }
    ">Upload</button>

    <h3 id="result"></h3>

</body>
</html>
"""

# Save webpage
webpage = Path("file_upload.html")
webpage.write_text(html, encoding="utf-8")
```

1923163

```
# Open Chrome browser
driver = webdriver.Chrome()

try:
    driver.get(webpage.resolve().as_uri())

    # Select file using Selenium
    driver.find_element(By.ID, "file").send_keys(str(sample_file))

    # Click Upload button
    driver.find_element(By.ID, "upload").click()

    # Get upload result
    result = driver.find_element(By.ID, "result").text

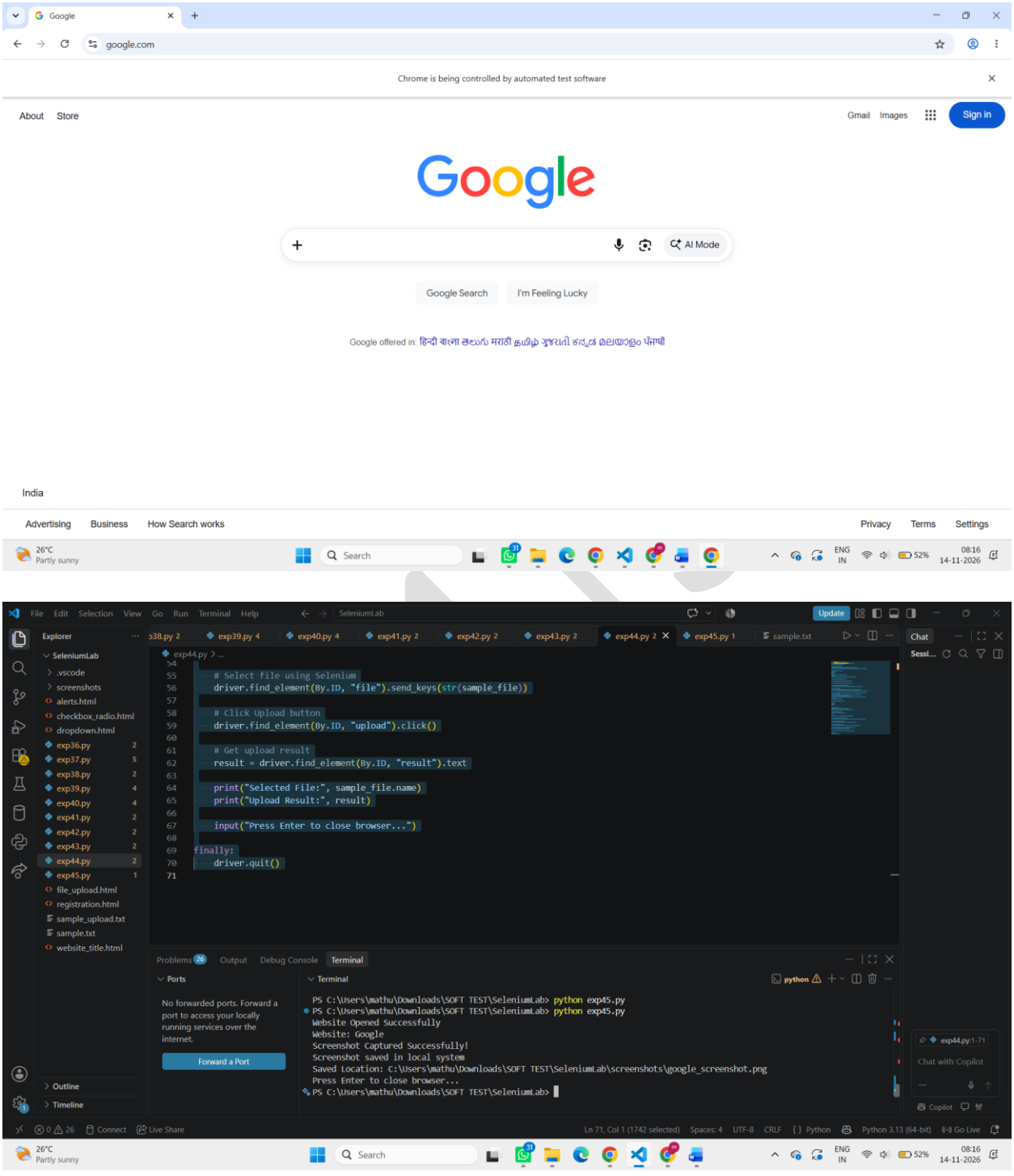
    print("Selected File:", sample_file.name)
    print("Upload Result:", result)

    input("Press Enter to close browser...")

finally:
    driver.quit()
```

1923163

OUTPUT



RESULT

Thus, a screenshot of the web page was captured and stored successfully in the local system.